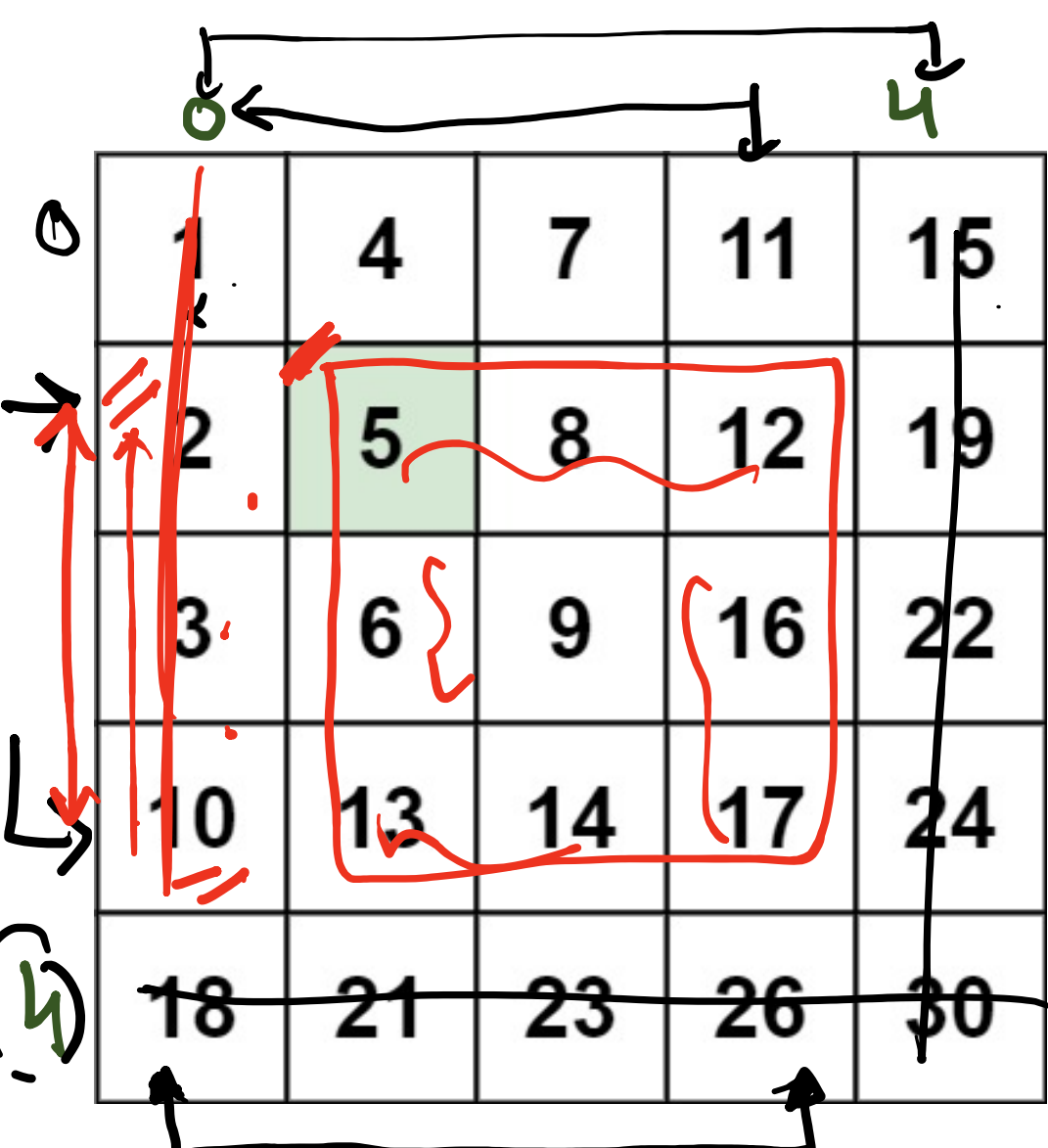


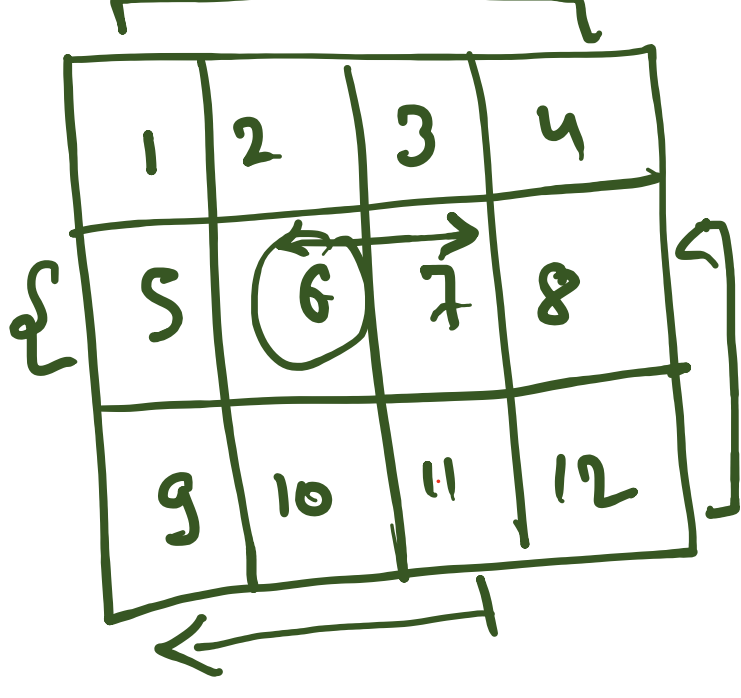
1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

1 4 7 11 15 19 22 24 30
2 6 23 21 18 10 3 2
5 8 12 16 17 14 13 6 9



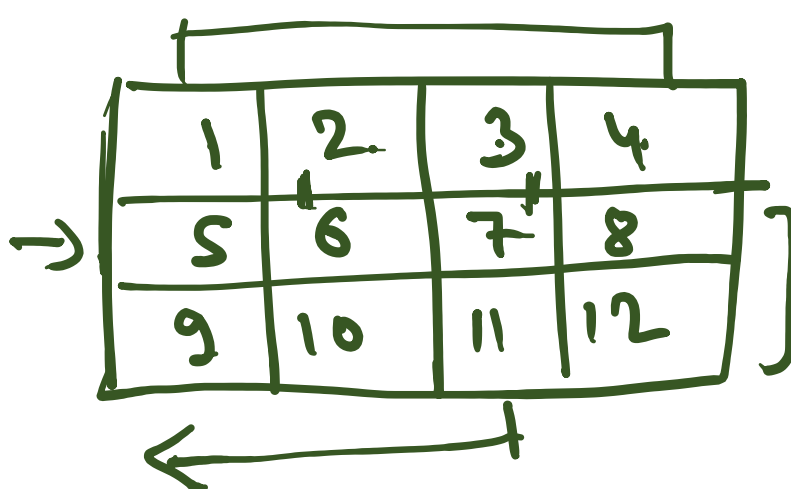
minR=0, minC=0, maxR=4, maxC=4
for (i = minC; i <= maxC; i++)
 for (j = minR; j <= maxR; j++)
 System.out.print(matrix[i][j] + " ");
 minR++;
for (i = minR; i <= maxR; i++)
 for (j = minC; j <= maxC; j++)
 System.out.print(matrix[i][j] + " ");
 maxC--;
for (i = maxC; i >= minC; i--)
 for (j = minR; j <= maxR; j++)
 System.out.print(matrix[i][j] + " ");
 maxR--;
for (i = maxR; i >= minR; i--)
 for (j = minC; j <= maxC; j++)
 System.out.print(matrix[i][j] + " ");
 minC++;

```
public static void Print(int[][] matrix) {  
    int minr = 0, maxr = matrix.length - 1, minc = 0, maxc = matrix[0].length - 1;  
    int te = matrix.length * matrix[0].length; 12  
    int c = 0;  
    while (c < te) {  
        for (int i = minc; i <= maxc; i++) {  
            System.out.print(matrix[minr][i] + " ");  
            c++;  
        }  
        minr++;  
        for (int i = minr; i <= maxc; i++) {  
            System.out.print(matrix[i][maxc] + " ");  
            c++;  
        }  
        maxc--;  
        for (int i = maxc; i >= minc; i--) {  
            System.out.print(matrix[maxr][i] + " ");  
            c++;  
        }  
        maxr--;  
        for (int i = maxr; i >= minr; i--) {  
            System.out.print(matrix[i][minc] + " ");  
            c++;  
        }  
        minc++;  
    }  
}
```

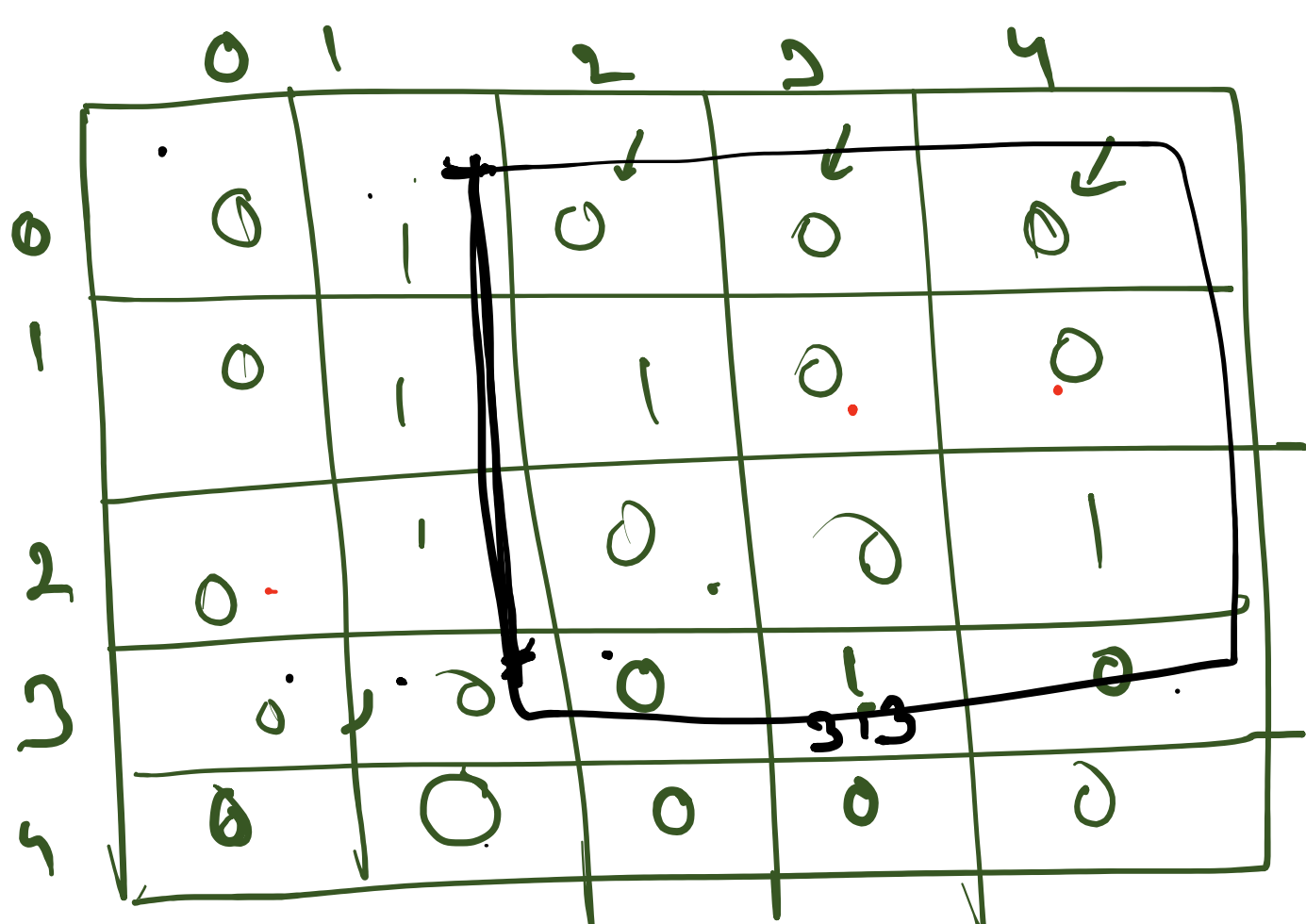


1 2 3 4 8 12
11 10 9 5 6 7 6

```
public static void Print(int[][] matrix) {  
    int minr = 0, maxr = matrix.length - 1, minc = 0, maxc = matrix[0].length - 1;  
    int te = matrix.length * matrix[0].length; 12  
    int c = 0;  
    while (c < te) {  
        for (int i = minc; i <= maxc && c < te; i++) {  
            System.out.print(matrix[minr][i] + " ");  
            c++;  
        }  
        minr++;  
        for (int i = minr; i <= maxc && c < te; i++) {  
            System.out.print(matrix[i][maxc] + " ");  
            c++;  
        }  
        maxc--;  
        for (int i = maxc; i >= minc && c < te; i--) {  
            System.out.print(matrix[maxr][i] + " ");  
            c++;  
        }  
        maxr--;  
        for (int i = maxr; i >= minr && c < te; i--) {  
            System.out.print(matrix[i][minc] + " ");  
            c++;  
        }  
        minc++;  
    }  
}
```



1 2 3 4 8 12
11 10 9 5 6 7 6



(3-0+1) x (4-1+1)
4 x 4 = 16

minR=0, maxR=4, minC=0, maxC=4

minR = min(minR, i)
maxR = max(maxR, i)
minC = min(minC, j)
maxC = max(maxC, j)

There are n gas stations along a circular route, where the amount of gas at the i^{th} station is $gas[i]$.

You have a car with an unlimited gas tank and it costs $cost[i]$ of gas to travel from the i^{th} station to its next $(i + 1)^{\text{th}}$ station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and $cost$, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

Input: $gas = [1,2,3,4,5]$, $cost = [3,4,5,1,2]$

Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 0 + 4 = 4

Travel to station 4. Your tank = 4 - 1 + 5 = 8

Travel to station 0. Your tank = 8 - 2 + 1 = 7

Travel to station 1. Your tank = 7 - 3 + 2 = 6

Travel to station 2. Your tank = 6 - 4 + 3 = 5

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

```
int[] gas = { 1, 2, 3, 4, 5 };  
int[] cost = { 3, 4, 5, 1, 2 };
```

idx = 0, 1, 2, 3

currsum = 0

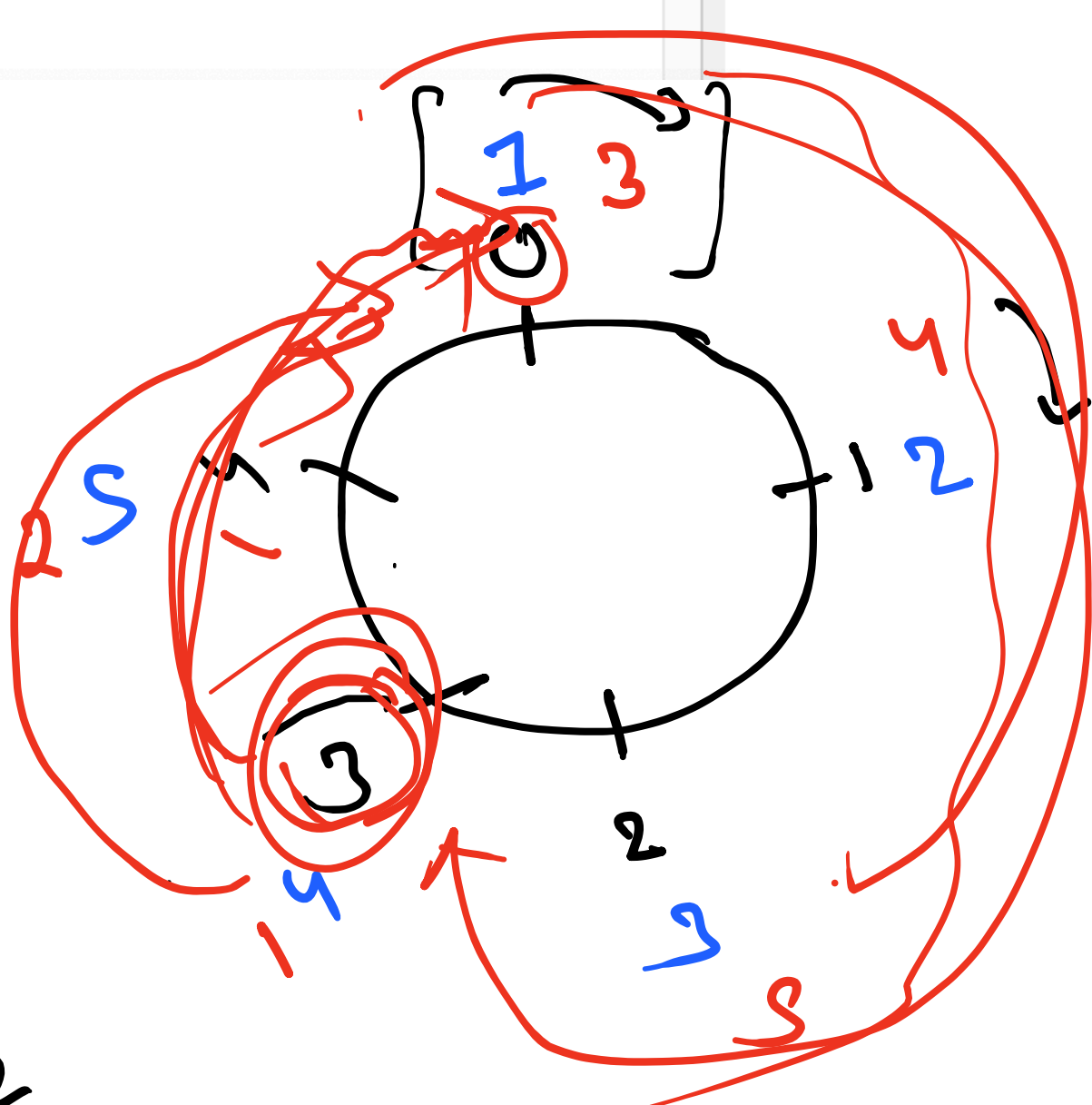
currsum += (1-3) = -2

currsum += (2-4) = -2

currsum += (3-5) = 2

currsum += (4-1) = 3

currsum += (5-2) = 6



gas sum - cost sum